

Just-in-Time Scheduling in Two-Stage Flexible Flow Shops

Yuval Itzhaki

Claude

Abstract

In the two-stage flexible flow shop $FF(1, m) \parallel \sum_j w_j Z_j$ every job is first preprocessed on a single machine and then processed on one of m identical machines, and the objective is the total weight of the jobs whose second operation ends exactly at their due date. Heeger, Hermelin, Itzhaki, Schieber and Shabtay [5] characterize the sets of jobs that can all be completed just in time, give five algorithms built on the characterization, and show that the problem is strongly NP-hard, and W[2]-hard for the number of machines, already for unit weights. The formalization verifies the scheduling algorithms as word RAM programs and the reductions from HITTING SET, including their correctness and complexity bounds. The hardness consequences follow from the corresponding hardness of HITTING SET; its NP-hardness is also formalized. This note presents the scheduling results in the notation of [5], with links to Lean. For completeness, the accompanying archive includes a supporting W-hierarchy development, which goes beyond the scope of this scheduling project.

1 Introduction

In a two-stage flexible flow shop every job is first preprocessed, without preemption, on a single first-stage machine and then processed on one of m identical second-stage machines. Job j has a preprocessing time p_j , a processing time q_j , a due date d_j and a weight w_j . It is *just in time* if its second operation ends exactly at d_j , and the goal is a schedule maximizing the total weight of the just-in-time jobs. In the three-field notation of Graham et al. the problem is $FF(1, m) \parallel \sum_j w_j Z_j$. The paper [5] characterizes the sets of jobs that can be completed just in time (Section 2), and builds on this characterization:

- a dynamic program in $O(W \cdot n^m)$ time, where $W = \sum_j w_j$ (Theorem 2), and in $O(P \cdot n^m)$ time, where $P = \sum_j p_j$ (Corollary 2);
- two further programs in $O(W \cdot 2^\omega \cdot n)$ and $O(W \cdot m^{q_{\max}} \cdot n)$ time, where ω is the largest number of jobs whose second operations overlap at one instant and $q_{\max} = \max_j q_j$ (Theorem 3);
- for equal preprocessing times, a program in $O(n^{m+1})$ time (Corollary 3), a greedy in $O(n \log n)$ time for unit weights, and an integer program with a totally unimodular matrix for the weighted case on proper instances (Theorem 4);
- a fully polynomial-time approximation scheme when m , ω or $q_{\max}$ is bounded (Theorem 5);
- strong NP-hardness, and W[2]-hardness for m , already for unit weights, by a reduction from HITTING SET (Theorem 1, Corollary 4).

Running times are statements about word RAM programs: a program is a fixed list of instructions operating on words of w bits, and its running time is the number of instructions it executes. Each algorithm of the paper is written as such a program in the IMP+ language of the archive, compiled by its verified compiler, and shown to compute the stated function within the stated number of instructions. The due dates and weights are entries of the input word, so a bound cannot be stated against the number of jobs alone: the entries must fit in words of length w , and this is part of every statement below.

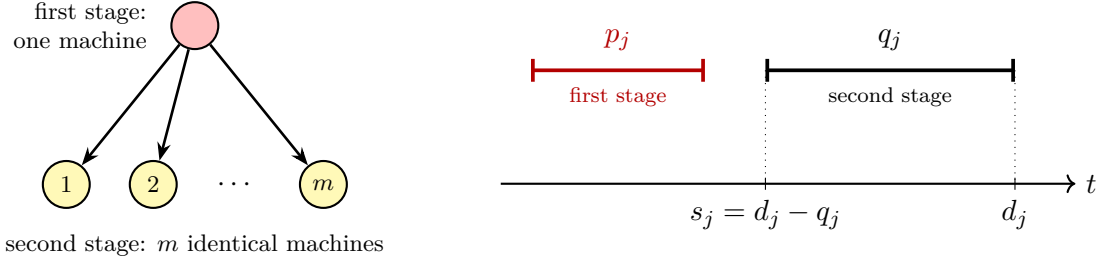


Figure 1: A flexible flow shop with one first-stage machine and m identical second-stage machines, and the two operations of a job j : the preprocessing, of length p_j , and the processing, which must occupy the interval $[s_j, d_j]$.

2 The Problem

Definition 1 ($FF(1, m) \parallel \sum_j w_j Z_j$). An instance consists of n jobs $\mathcal{J} = \{1, \dots, n\}$ and m identical second-stage machines. Job j has a preprocessing time $p_j \in \mathbb{N}$ on the single first-stage machine, a processing time $q_j \in \mathbb{N}$ on a second-stage machine, a due date $d_j \in \mathbb{N}$ and a weight $w_j \in \mathbb{N}$. The two operations of a job are run in this order and without preemption. Job j is *completed just in time* if its second operation ends exactly at d_j , and the objective is to maximize the total weight of the jobs completed just in time.

Completing job j just in time pins its second operation to the interval $[s_j, d_j]$, where $s_j = d_j - q_j$ is its *start time*; the first operation must be finished by s_j . Two jobs are *in conflict* if their intervals overlap, and conflicting jobs cannot share a second-stage machine. Only the jobs completed just in time matter, so a solution is the set $\mathcal{Z}$ of these jobs, and $\mathcal{Z}$ is *feasible* if there is a schedule completing every job of $\mathcal{Z}$ just in time. See Figure 1.

Jobs are indexed in *earliest-start-time order* (EST order), so that $s_1 \leq \dots \leq s_n$; as in the paper, processing times are positive. The programs of Sections 4 and 5 also assume that the $2n$ endpoints s_j, d_j are pairwise distinct, which a rescaling achieves (Section 4).

A set $\mathcal{Z} \subseteq \mathcal{J}$ satisfies

- *Condition 1* if it can be preprocessed in time: $\sum_{i \in \mathcal{Z}, i \leq j} p_i \leq s_j$ for every $j \in \mathcal{Z}$; and
- *Condition 2* if it can be scheduled on m machines: $\mathcal{Z}$ can be partitioned into m classes, each without two jobs in conflict.

Characterization. A set $\mathcal{Z}$ is feasible if and only if it satisfies Conditions 1 and 2. Condition 2 holds if and only if at most m jobs of $\mathcal{Z}$ are running at any one instant.

The two stages decouple: the first-stage machine and the second-stage machines constrain $\mathcal{Z}$ separately, and schedules meeting the two conditions combine into one schedule of the shop. The second sentence is the perfectness of interval graphs; it is what allows the tables of Sections 4 and 5 to be indexed by the jobs alive at an instant.

Observation 1. A feasible set has a just-in-time schedule in which the first-stage operations are run in EST order.

By the exchange argument of Moore [8]: if the first stage can meet the deadlines s_j in some order, it can meet them in order of nondecreasing s_j . Hence nothing is lost by deciding which jobs to select without also deciding in which order to preprocess them. Figure 2 shows a feasible set of five jobs.

An instance is given to a word RAM as a word of numbers: n, m , the n preprocessing times, the n processing times, the n due dates and the n weights, in this order. In the decision version the word ends with the threshold W^* . For NP-hardness an instance is encoded as a binary word.

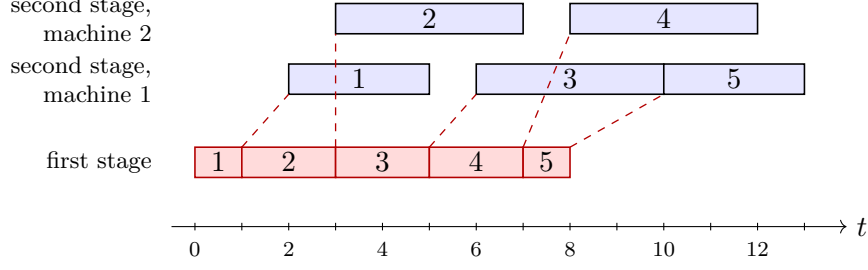


Figure 2: A feasible set of five jobs on two second-stage machines. The first stage runs the jobs in EST order, and the preprocessing of job j ends by s_j (dashed). The jobs on each machine do not overlap.

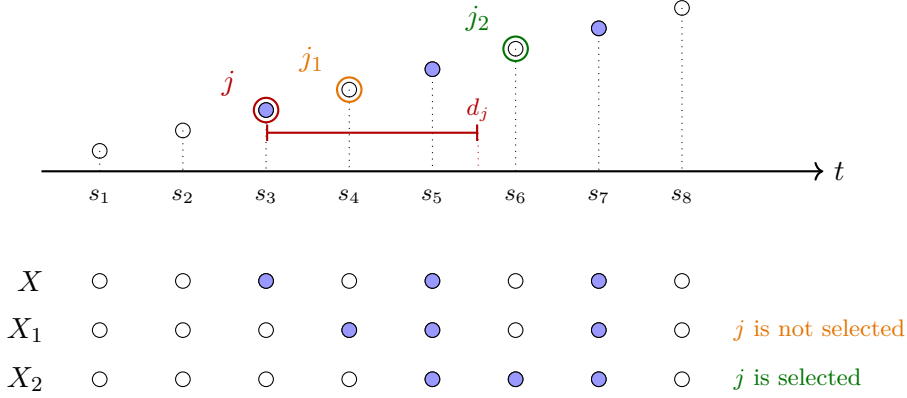


Figure 3: The thresholds $X = \{3, 5, 7\}$ among eight jobs in EST order, with $j = 3$. The set X_1 replaces j by the smallest index $j_1 = 4$ outside X above it; the set X_2 replaces it by the smallest index $j_2 = 6$ outside X whose start time is at least d_j . Recursion (1) reads the table at X_1 if j is not selected and at X_2 if it is.

We consider the decision problem of whether there is a feasible set of weight at least W^* , parameterized by the number m of machines. The quantities W , ω , $q_{\max}$ and P that appear in the running times are functions of the word. The bounds below are those of the paper with a term $O(n \log n)$ added for putting the jobs into EST order, which the paper assumes done and a program reading an arbitrary word has to pay for.

3 A Fixed Number of Machines

The jobs are considered from the last to the first. A set $X \subseteq \mathcal{J}$ of at most m thresholds describes the second-stage machines: a schedule is *compatible* with $X = \{x_1 < \dots < x_{|X|}\}$ if machine i runs only jobs from $\{x_i, \dots, n\}$ and the machines $|X| + 1, \dots, m$ run none. The entry $T[X, W']$ of the table is the largest P' such that some feasible $\mathcal{Z}$ of weight W' compatible with X can be preprocessed starting at time P' ; it is $-\infty$ if there is none and $+\infty$ if the empty set will do. For $X \neq \emptyset$ let $j = \min X$, let $j_1 = \min\{j^* \notin X : j^* > j\}$ and $j_2 = \min\{j^* \notin X : s_{j^*} \geq d_j\}$, and let $X_i = (X \setminus \{j\}) \cup \{j_i\}$, or $X \setminus \{j\}$ if j_i does not exist (Figure 3). The table is filled by

$$T[X, W'] = \max \begin{cases} T[X_1, W'], \\ \min\{T[X_2, W' - w_j], s_j\} - p_j. \end{cases} \quad (1)$$

Lemma 1. *Recursion (1) correctly computes $T[X, W']$ for every X with $|X| \leq m$ and every W' .*

The recursion is formalized as an equivalence: a set of weight W' compatible with X can be preprocessed from P' exactly when a set of weight W' compatible with X_1 can (job j is not selected), or $w_j \leq W'$, $P' + p_j \leq s_j$, and a set of weight $W' - w_j$ compatible with X_2 can be preprocessed from $P' + p_j$ (job j is selected and preprocessed first). The table is the predicate “can be preprocessed from P' ”, which is downward closed in P' and so needs no infinite values. The hard direction takes a solution in which j is not selected and reassigns its jobs to the thresholds of X_1 , an application of Hall’s theorem.

Theorem 2. $FF(1, m) \parallel \sum_j w_j Z_j$ is solvable in $O(W \cdot n^m)$ time.

The answer is read off the table at the m smallest indices: a set of weight W' compatible with them that can be preprocessed from time 0 is exactly a feasible set of weight W' . The formalized statement is a word RAM program, together with a constant c , that decides the problem on every instance x with threshold W^* , at every word length w at which the input and the table fit, within $c(W^* + 1)(n + 1)^m$ instructions plus the cost of sorting. The table has $O(n^m)$ columns and $W + 1$ rows, each entry computed from two earlier ones.

Corollary 1. $FF(1, 1) \parallel \sum_j Z_j$ is solvable in $O(n^2)$ time.

This is Theorem 2 with $m = 1$ and $W = n$.

Corollary 2. $FF(1, m) \parallel \sum_j w_j Z_j$ is solvable in $O(P \cdot n^m)$ time, where $P = \sum_j p_j$.

The same table is read the other way round: it stores for each instant $P' \in \{0, \dots, P\}$ the largest weight attainable, by the recursion $T[X, P'] = \max\{T[X_1, P'], T[X_2, P' + p_j] + w_j\}$, where the second term is $-\infty$ if $P' + p_j > s_j$.

4 A Program for Small ω

Setting $d'_j = (n + 1)d_j + j$, $q'_j = (n + 1)q_j$ and $p'_j = (n + 1)p_j$ gives an equivalent instance with pairwise distinct endpoints: it has the same feasible sets, with the same weights, and it preserves the EST order. The content is that the rescaling creates no conflict where there was none and destroys none where there was one.

Let $t_1 < \dots < t_{2n}$ be the endpoints and $X_i = \{j : s_j \leq t_i < d_j\}$ the set of jobs whose second operation contains t_i ; so $\omega = \max_i |X_i|$. Let $\mathcal{J}_i = \bigcup_{\ell \leq i} X_\ell$. A feasible $\mathcal{Z} \subseteq \mathcal{J}_i$ is *compatible* with $X \subseteq X_i$ if $\mathcal{Z} \cap X_i = X$, and

$T_i[X, W'] =$ the least $p(\mathcal{Z})$ over feasible $\mathcal{Z} \subseteq \mathcal{J}_i$ of weight W' compatible with X .

Since $X \subseteq X_i$ has at most ω members, each table has $W \cdot 2^\omega$ entries. If $t_i = s_j$, then

$$T_i[X, W'] = \begin{cases} T_{i-1}[X, W'] & j \notin X, \\ T_{i-1}[X \setminus \{j\}, W' - w_j] + p_j & j \in X \text{ and this is at most } s_j, \\ \infty & \text{otherwise,} \end{cases} \quad (2)$$

and if $t_i = d_j$, then

$$T_i[X, W'] = \begin{cases} T_{i-1}[X, W'] & |X| = m, \\ \min\{T_{i-1}[X \cup \{j\}, W'], T_{i-1}[X, W']\} & |X| < m. \end{cases} \quad (3)$$

Lemma 2. Recursions (2) and (3) correctly compute $T_i[X, W']$ for every i , every $X \subseteq X_i$ and every W' .

Both equations are formalized as equivalences, so each says at once that the recursion invents no entry and loses none. Past the last start time the table answers the question: some state carries weight W' exactly when a feasible set of weight W' exists.

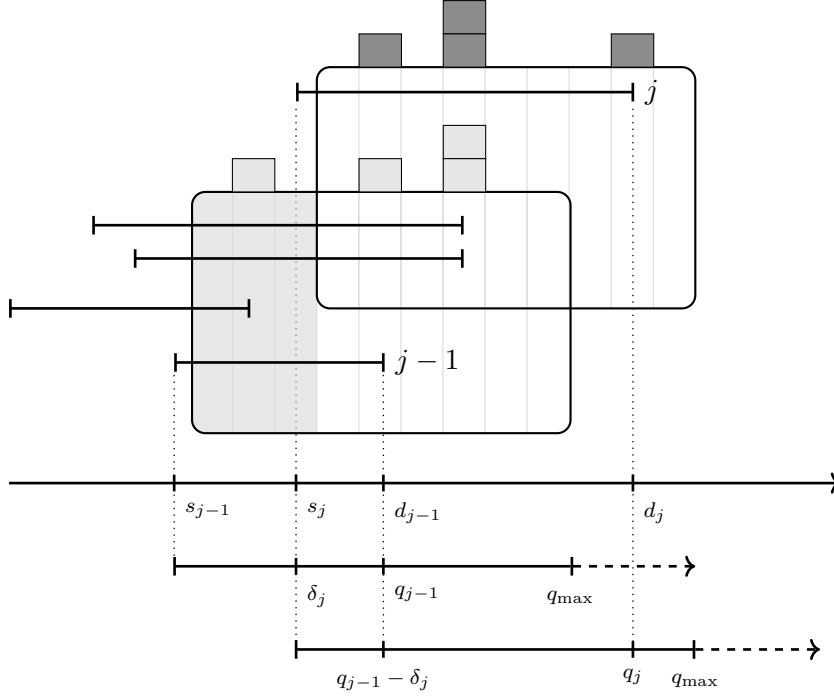


Figure 4: The dynamic program visualized. A set $\mathcal{Z}$ of jobs is shown in three reference frames: the top axis is the main line beginning at 0, the middle axis the reference of $j - 1$ beginning at s_{j-1} , and the bottom one the reference of j beginning at s_j . The boxes above each sliding window represent the number of machines that complete processing their last job at that time. In the example, assuming that all jobs can be preprocessed in time, $\mathcal{Z}$ has profile $\vec{x}$ at s_j and the solution $\mathcal{Z} \setminus \{j\}$ has profile $\vec{y}$ at s_{j-1} , with $q_{\max} = 9$, $\delta_j = 3$, $\vec{x} = (0, 1, 0, 2, 0, 0, 0, 1, 0)$ and $\vec{y} = (0, 1, 0, 0, 1, 0, 2, 0, 0)$. Observe that $\vec{y} \in \vec{x}^{(q_j)}[\delta_j]$, and also $\vec{y} \in \vec{x}[\delta_j]$; in the example this set is $\vec{x}^{(8)}[3]$.

5 A Program for Small $q_{\max}$

The second program sweeps the jobs in EST order and carries, instead of the selected jobs alive at s_j , only their *profile*: the vector $\vec{x} \in \{0, \dots, m\}^{q_{\max}}$ whose i th entry is the number of machines that complete their last selected job at $s_j + i$, equivalently the number of selected jobs due at $s_j + i$. Stepping from s_{j-1} to s_j moves the reference point by $\delta_j = s_j - s_{j-1}$, so a profile at s_{j-1} becomes a profile at s_j shifted by δ_j ; let $\vec{x}[\delta]$ be the set of vectors that shift to $\vec{x}$, and $\vec{x}^{(q)}$ the vector with entry q decreased by one. The table $T_j[\vec{x}, W']$ is the least $p(\mathcal{Z})$ over feasible selections $\mathcal{Z} \subseteq \{1, \dots, j\}$ of weight W' whose profile at s_j is $\vec{x}$, and

$$T_j[\vec{x}, W'] = \min \begin{cases} \min_{\vec{y} \in \vec{x}[\delta_j]} T_{j-1}[\vec{y}, W'], \\ \min_{\vec{y} \in \vec{x}^{(q_j)}[\delta_j]} \begin{cases} T_{j-1}[\vec{y}, W' - w_j] + p_j & \text{if this is at most } s_j, \\ \infty & \text{otherwise.} \end{cases} \end{cases} \quad (4)$$

The first case passes job j over, the second selects it (Figure 4).

Lemma 3. *Recursion (4) computes the table of due-date profiles.*

The formalization maintains the invariant in the following form: an entry $T_j[\vec{x}, W'] \leq P'$ is witnessed by a feasible selection of weight W' and preprocessing cost at most P' whose profile is *at most* $\vec{x}$ coordinatewise, and every feasible selection is still derived with its exact profile. A vector that overstates the profile only overstates how many machines are busy, so the capacity test rejects more rather than less, and the optimum read off at the end is exact. Small examples,

one job on two machines and two jobs on one machine, show that the entries do not carry the exact profile, which is why the weaker form is the one that is proved.

Theorem 3. $FF(1, m) \parallel \sum_j w_j Z_j$ can be solved in

- $O(W \cdot 2^\omega \cdot n)$ time, where ω is the largest number of jobs whose second operations contain a common instant, and
- $O(W \cdot m^{q_{\max}} \cdot n)$ time, where $q_{\max} = \max_j q_j$.

The first bullet is a word RAM program that carries the table of recursions (2) and (3) through the $2n$ endpoints. The second is a program that carries the profiles of recursion (4), one marginalization pass per job. Each bound has its own fitting condition, for its own table, and neither involves m in the exponent.

6 Uniform Preprocessing Times

In this section $p_j = p$ for all jobs. An instance is *proper* if no interval $[s_i, d_i]$ is contained in another; the EST order and the order by due dates then coincide.

For unit weights the jobs are taken in EST order, maintaining a set $\mathcal{Z}$ of selected jobs. At job j , if $s_j \geq (|\mathcal{Z}| + 1)p$ and fewer than m jobs of $\mathcal{Z}$ are alive at s_j , then j joins $\mathcal{Z}$; otherwise one job of $\mathcal{Z} \cup \{j\}$ with the largest due date is dropped. The set kept is compared to others through a *domination* order: $\mathcal{Z}'$ dominates $\mathcal{Z}''$ if it has more jobs, or the same number and, for every i , its i th largest due date is not greater than the i th largest due date of $\mathcal{Z}''$.

Lemma 4. *After the greedy has considered the first j jobs, its set is feasible and dominates every feasible set of jobs among them. Consequently it ends with a feasible set of largest cardinality.*

The proof is by induction on j over the three cases of the step: the job joins the set, or the first stage cannot preprocess one more job, or m selected jobs are alive at s_j . In the last two the greedy keeps the set of the previous step without its latest due date.

For weights, on a proper instance, a set is feasible exactly when its indicator vector $\mathbf{x} \in \{0, 1\}^n$ satisfies, for every job j ,

$$\sum_{i \leq j} x_i \leq \lfloor s_j/p \rfloor, \quad \sum_{i: s_i \leq s_j < d_i} x_i \leq m. \quad (5)$$

The first inequality is Condition 1, since with equal preprocessing times the time spent is the number of jobs run, and the second is Condition 2 in its depth form, tested at the start times. The scheduling problem is therefore the integer program that maximizes $\sum_j w_j x_j$ subject to (5).

A matrix has the *consecutive ones property* if the ones in each row are consecutive. Such a matrix is totally unimodular [3]: every square submatrix has determinant 0, 1 or -1 . This theorem of Fulkerson and Gross is proved rather than assumed.

Lemma 5. *On a proper instance the constraint matrix of (5) has the consecutive ones property, and hence is totally unimodular.*

The jobs alive at any one instant form a consecutive block of the EST order, so the first family of rows consists of prefixes and the second of blocks. That the feasible solutions of the program are exactly the feasible sets holds on every instance with equal positive preprocessing times, proper or not.

Corollary 3. $FF(1, m) \mid p_j = p \mid \sum_j w_j Z_j$ is solvable in $O(n^{m+1})$ time.

With equal preprocessing times the instant a partial solution has spent is determined by the number of jobs it has selected, so the dual table of Corollary 2 takes only $n + 1$ values of P' .

Theorem 4. $FF(1, m) \mid p_j = p \mid \sum_j w_j Z_j$ is solvable in $O(n \log n)$ time when $w_j = 1$ for all jobs, and in $O(n^{2.5} \cdot \max\{W, \log P\})$ time on proper instances.

The first part is the greedy, whose cost beyond sorting is one pass. The formalized part of the second is the integer program, its correctness and the total unimodularity of its matrix (Lemma 5); that a linear program with a totally unimodular matrix has an integral optimum [6] and is solved in polynomial time are cited.

7 Approximation Schemes

Let $\varepsilon > 0$ and $k = \varepsilon w_{\max}/n$, and round the weights to $w_j^* = \lceil w_j/k \rceil$. Let $\mathcal{Z}^*$ be an optimal solution for w^* and $\mathcal{Z}$ one for w . The optimum is at least $w_{\max}$, so

$$w(\mathcal{Z}^*) \geq k w^*(\mathcal{Z}^*) - kn \geq w(\mathcal{Z}) - \varepsilon w_{\max} \geq (1 - \varepsilon) w(\mathcal{Z}). \quad (6)$$

An *approximation scheme* is a program that is handed an instance and an integer e and returns a number that a feasible set reaches and that is within a factor $1 - 1/e$ of the optimum.

Theorem 5. $FF(1, m) \parallel \sum_j w_j Z_j$ admits a fully polynomial-time approximation scheme when $m = O(1)$, or $q_{\max} = O(1)$, or $\omega = O(\log n)$.

The rounding argument (6) is proved on its own. It assumes that there is a machine to run a job on and that $p_j \leq s_j$ for every job, jobs failing which are discarded. After rounding, the threshold to search is of order $n^2 e$, so each of the three exact programs becomes a scheme whose running time is that of the program with W replaced by $n^2 e$: the table of Theorem 2 gives the scheme for $m = O(1)$, the endpoint sweep gives the scheme for $\omega = O(\log n)$, and the profile sweep gives the scheme for $q_{\max} = O(1)$.

8 Hardness

Definition 2 (HITTING SET). We are given a family $\mathcal{F} = \{F_1, \dots, F_m\}$ of subsets of a universe $U = \{1, \dots, n\}$ and an integer k , and are asked whether there is a set $H \subseteq U$ with $|H| = k$ that meets every $F \in \mathcal{F}$. The parameter is k .

HITTING SET is NP-hard [4] (problem SP8) and W[2]-hard for k [2]. The instances considered here have $2 \leq k \leq n$.

A problem is NP-hard if every language in NP has a polynomial-time many-one reduction to it, and *strongly* NP-hard if the reduction can be chosen to emit instances whose numbers are bounded by a fixed polynomial in the length of the input. An *fpt-reduction* is a map on words that preserves and reflects yes-instances, raises the parameter by at most a function of it, and is computed by a word RAM program within $c g(k)(|x| + 1)^c$ instructions. A parameterized problem is W[2]-hard if HITTING SET, parameterized by k , fpt-reduces to it.

Construction 1. Let $(U, \mathcal{F}, k)$ be an instance of HITTING SET with $2 \leq k \leq n$. Let $R = k(n - 1) + 2$ be the number of *segments* and $Q = (k - 1)(n + 1)$. We build an instance of $FF(1, k) \parallel \sum_j Z_j$. The time axis is divided into R segments of m *epochs* each, one epoch per set of the family. For $r \in \{0, \dots, R - 1\}$ and $j \in \{1, \dots, m\}$ let $g(r, j) = rm + j$ and $G(r, j) = g(r, j)^2 Q$; the epoch T_j^r is the interval from $G(r, j)$ to the start of the next epoch, of length $(2g(r, j) + 1)Q$. The jobs of epoch T_j^r , all of weight 1, are

- for each $i \in F_j$ a *selection job* $S_{i,j}^r$ with preprocessing time 0, processing time $(2g(r, j) + 1)Q$ and due date $G(r, j) + (2g(r, j) + 1)Q + i$;
- for each $i \in U$ a *dummy job* $A_{i,j}^r$ with preprocessing time $g(r, j)(n + 1)$, processing time $g(r, j)Q$ and due date $G(r, j) + g(r, j)Q + i$; and

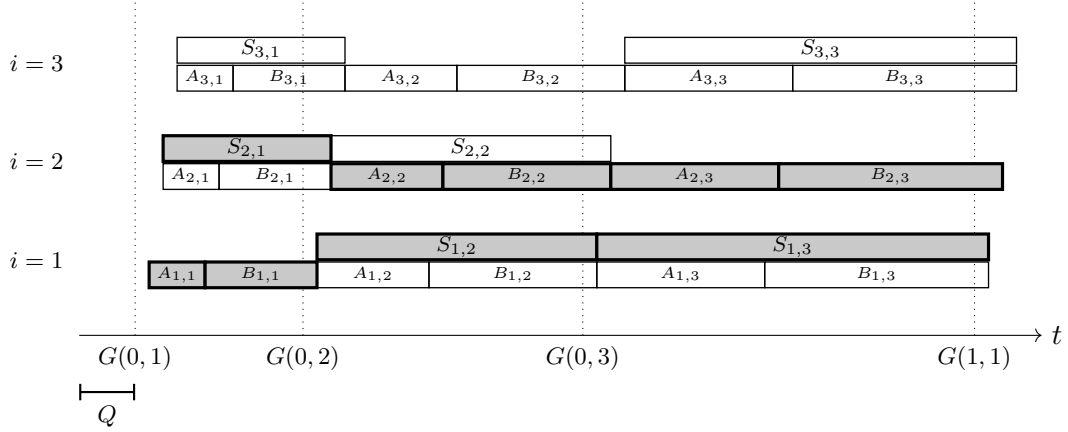


Figure 5: Construction 1 for $U = \{1, 2, 3\}$, $F_1 = \{2, 3\}$, $F_2 = \{1, 2\}$, $F_3 = \{1, 3\}$ and $k = 2$, drawn to scale; only the first segment is shown. Each row holds the jobs of one element i , shifted by i , with the selection jobs above the dummy pairs. Gray jobs form the schedule for the hitting set $H = \{1, 2\}$: in each epoch one machine runs a selection job and the other runs the two dummy jobs of its element.

- for each $i \in U$ a *dummy job* $B_{i,j}^r$ with preprocessing time $g(r, j)(n + 1)$, processing time $(g(r, j) + 1)Q$ and due date $G(r, j) + (2g(r, j) + 1)Q + i$.

The target is $R \cdot m \cdot (2k - 1)$ just-in-time jobs.

Thus $A_{i,j}^r$ ends when $B_{i,j}^r$ starts, and $B_{i,j}^r$ ends when the next epoch starts, up to the offset i ; see Figure 5. Two details are done slightly differently than in [5]: the number of segments is $k(n - 1) + 2$, which is what the pigeonhole argument below uses, and the construction is stated for $2 \leq k \leq n$, the range in which Q is positive and a hitting set of size k can exist. The elements are numbered from 0 in the formalization, with $i + 1$ in place of i in the due dates.

Theorem 1. $FF(1, m) \parallel \sum_j Z_j$ is strongly NP-hard.

The family has a hitting set of size k if and only if the instance of Construction 1 has $R \cdot m \cdot (2k - 1)$ jobs that can be completed just in time. Given a hitting set H , let i_j be the least element of $F_j \cap H$. In every epoch the machine of i_j runs the selection job $S_{i_j,j}^r$, and the machine of every other element of H runs its two dummy jobs; the preprocessing of the $k - 1$ dummy pairs of an epoch takes exactly $2g(r, j)Q$ and fits before the epoch starts. Conversely, the preprocessing budget of an epoch admits at most $2(k - 1)$ dummy jobs, so a solution of the target size holds exactly one selection job and $2(k - 1)$ dummies in every epoch. The offsets i make the element a machine is responsible for nondecreasing from one segment to the next, and it takes values in $\{1, \dots, n\}$, so it changes at most $n - 1$ times. With more than $k(n - 1)$ gaps between segments some gap sees no change on any machine, and the k elements of that segment hit every set. The numbers of the instance are polynomial in n , m and k , hence in the length of the HITTING SET instance, so the hardness is strong: no pseudo-polynomial algorithm exists unless $P = NP$.

Corollary 4. $FF(1, m) \parallel \sum_j Z_j$ is $W[2]$ -hard with respect to the number m of machines.

The shop of Construction 1 has exactly k machines, so the construction is an fpt-reduction from HITTING SET. This existence of a reduction is the formal statement. Together with the $W[2]$ -hardness of HITTING SET, it yields the stated hardness consequence; an algorithm with running time $f(m) \cdot n^{O(1)}$ would therefore give $FPT = W[2]$.

Theorem 1 rests on the NP-hardness of HITTING SET, which is proved here as well, by the standard reduction from satisfiability. A CNF formula F on variables $x_0, \dots, x_{V-1}$ becomes

the instance on the universe $\{0, \dots, 2V - 1\}$ in which $2i$ stands for the literal x_i and $2i + 1$ for $\bar{x}_i$. The family has one set $\{2i, 2i + 1\}$ for each variable and one set for each clause, holding the elements of its literals, and $k = V$. A hitting set of size V meets each of the V disjoint pairs in exactly one element, so it is a truth assignment, and it meets the set of a clause exactly when the assignment satisfies the clause. This is the composition of Karp's reductions [7] from satisfiability to Clique and from Clique to Node Cover, read on the literals.

Theorem (Karp). *Every language in NP reduces to HITTING SET in polynomial time, by a reduction whose output instances are of polynomial size and satisfy $2 \leq k \leq n$.*

A formula is satisfiable exactly when its instance has a hitting set of the required size. The reduction on words decodes a formula, builds the instance and encodes it; it is a word RAM program, and its polynomial running time is transferred to a Turing machine by the equivalence of the two machine models. Composed with the Cook–Levin theorem of the archive, this makes every language in NP reduce to HITTING SET. The two restrictions on k cost nothing: a hitting set of size exactly k cannot exist once k exceeds the universe, and for $k < 2$ an instance is padded with a fresh element and the singleton containing it, which forces that element into every hitting set and raises k by one. The universe of an emitted instance is no larger than $4 + m + \sum_j |F_j|$, so the shop of Construction 1, which has more than n^2 jobs, can be written in time polynomial in the word that presents the instance.

References

- [1] M. C. Carlisle and E. L. Lloyd. On the k -Coloring of Intervals. *Discrete Applied Mathematics* 59(3):225–235, 1995. DOI: [10.1016/0166-218X\(95\)80003-M](https://doi.org/10.1016/0166-218X(95)80003-M)
- [2] R. G. Downey and M. R. Fellows. *Parameterized Complexity*. Springer, 1999. DOI: [10.1007/978-1-4612-0515-9](https://doi.org/10.1007/978-1-4612-0515-9)
- [3] D. R. Fulkerson and O. A. Gross. Incidence Matrices and Interval Graphs. *Pacific Journal of Mathematics* 15(3):835–855, 1965. DOI: [10.2140/pjm.1965.15.835](https://doi.org/10.2140/pjm.1965.15.835)
- [4] M. R. Garey and D. S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.
- [5] K. Heeger, D. Hermelin, Y. Itzhaki, B. Schieber and D. Shabtay. Just-in-Time Scheduling in Two-Stage Flexible Flow Shops. *European Journal of Operational Research* 333:652–664, 2026. DOI: [10.1016/j.ejor.2026.02.017](https://doi.org/10.1016/j.ejor.2026.02.017)
- [6] A. J. Hoffman and J. B. Kruskal. Integral Boundary Points of Convex Polyhedra. In *Linear Inequalities and Related Systems*, pages 223–246. Princeton University Press, 1956.
- [7] R. M. Karp. Reducibility among Combinatorial Problems. In *Complexity of Computer Computations*, pages 85–103. Plenum Press, 1972. DOI: [10.1007/978-1-4684-2001-2_9](https://doi.org/10.1007/978-1-4684-2001-2_9)
- [8] J. M. Moore. An n Job, One Machine Sequencing Algorithm for Minimizing the Number of Late Jobs. *Management Science* 15(1):102–109, 1968. DOI: [10.1287/mnsc.15.1.102](https://doi.org/10.1287/mnsc.15.1.102)